

M13 : Environnement de Développement Intégré C#

Chapitre N°2. Introduction et généralités sur C# :

a. Définition général :

C# faisant partie de framework.Net est un produit Microsoft (créé en 2002) de développement d'applications informatiques.

C'est un langage de POO ayant une syntaxe proche du C++ et Java.

Un code C# est d'abord **précompilé**, cela donne lieu à un code **(.exe)** en langage intermédiaire **(CIL)**, ce dernier est encore compilé dans une machine virtuelle installée dans son ordinateur **CLR** (Common Language Runtime) pour aboutir en fin de compte un binaire exécutable.

Pour exploiter C#, il est intéressant d'utiliser l'IDE Visual C# 2010 (ou 2026) Express.

Il peut être couplé avec SQL Server 2008 express.

b. Le langage C# :

Il permet de développer tout type d'application exemples :

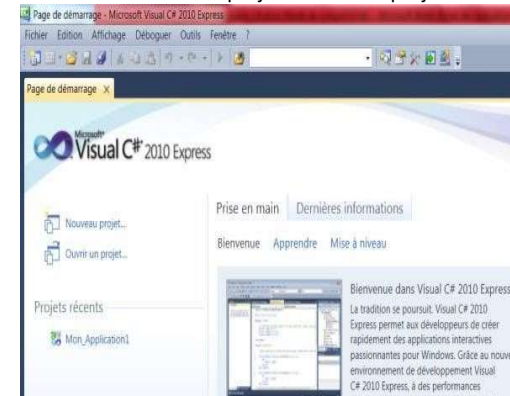
Développer des applications fenêtrées (**Windows Forms ou WPF**),

Des sites WEB (**ASP.net Web Forms et MVC**),

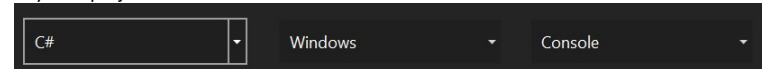
Se connecter aux bases de données (**ADO.net, LINQ et EntityFramework**).

c. Créer Un projet :

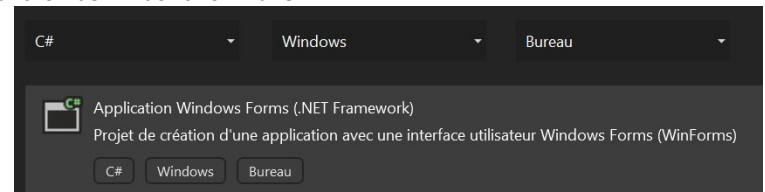
En lance l'IDE de Visual C# 2010 (ou 2026) Express présente la page de démarrage avec deux choix d'ouvrir un nouveau projet ou bien un projet existant.



Choix de style de projet dans IDE : **Pour choix de console**



Ou **Pour choix de Windows Formulaire**



المملكة المغربية
وزارة التربية الوطنية
والتعليم الأولي والابتدائي
الأكاديمية الجهوية للتربية والتكوين
جهة مكناس تضاير الحسبة
المديرية الإقليمية المراكش



المملكة المغربية
وزارة التربية الوطنية
والتعليم الأولي والابتدائي
الأكاديمية الجهوية للتربية والتكوين
جهة مكناس تضاير الحسبة
المديرية الإقليمية المراكش

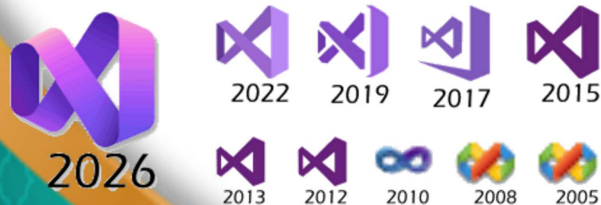


Lycée Moulay Mouhamed Ben AbdEllah
BREVET
TECHNICIEN
SUPÉRIEUR
Développement des Applications Informatiques
LARACHE

Module N°13

Environnement de Développement Intégré

Support de Cours, TP et Exercices



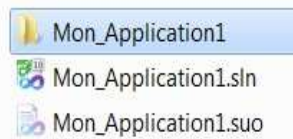
Prof : DOUFAKAR Said

M13





La création de nouveau projet permet d'avoir les fichiers suivants :



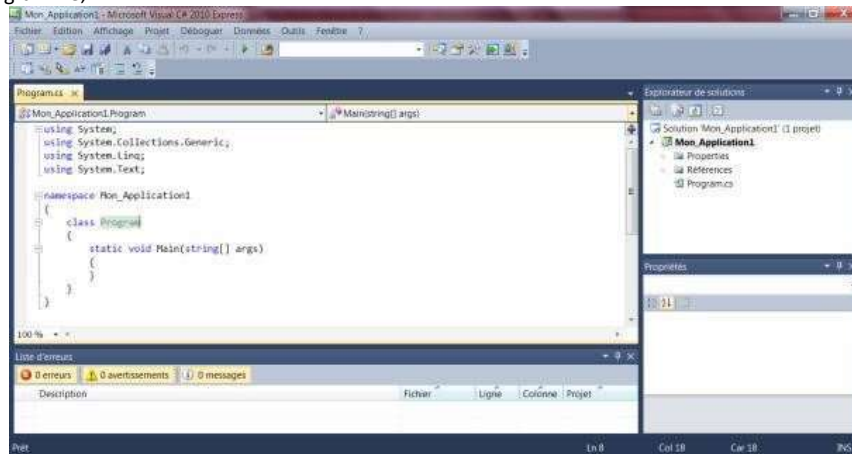
Mon_Application1 contient * et **

Mon_Application1.csproj (*): décrit le projet écrit en C#.

Program.cs ()** : contient le code source C#.

Mon_Application1.sln : fichier de solution, qui représente un container de projets permettant de les visualiser dans l'IDE.

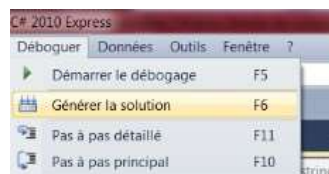
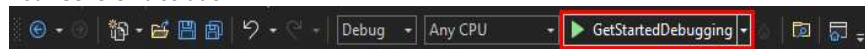
Le fichier « Program.cs » contient la méthode « **Main** » est la méthode principale du programme,



Exemple de la méthode Main() qui va permet d'afficher un message Bonjour:

```
static void Main(string[] args)
{
    Console.WriteLine("Bonjour");
}
```

Pour Générer la solution :



I. Les variables :

En C# les variables sont fortement typé.

Type	Description
Byte	Entier de 0 à 255 Byte i=10
Short	Entier de -32768 à 32767 short i=1000
Int	Entier de -2147483648 à 2147483647 int i=18
Long	Entier de -9223372036854775808 à 9223372036854775807
Float	Nombre simple précision de -3,4x10 ³⁸ à 3,4x10 ³⁸
Double	Nombre double précision de -1,79x10 ³⁰⁸ à 1,79x10 ³⁰⁸
Decimal	Nombre décimal utile pour les calculs financiers
Char	Un caractère
String	Une chaîne de caractères String x= "Hello"
Bool	Une valeur booléenne (true ou false)

II. Conversion de type en C# :

short petit = 10;	int grand = petit;
int grand = 25;	short petit = (short)grand;
int entier = 20;	double reel = entier;
double reel = 20.25;	int entier = (int)reel;
int entier = 20;	Convert.ToString (entier);
double reel = 20.55;	Convert.ToString (reel);
string texte = "20";	int entier = Convert.ToInt32 (texte);
string texte = "20";	int entier = int.Parse (texte);
string texte="20,25";	double reel = Convert.ToDouble (texte);
string texte = "20,25";	double reel = double.Parse (texte);

III. Lecture et écriture en C# :

a) Pour afficher les informations :

```
int i = 0;
double d = 10.25; string texte = "Bonjour"; Console.WriteLine(i.GetType());
Console.WriteLine(d.GetType());
Console.WriteLine(texte.GetType());
```

b) Pour saisir une information :

```
int n;
string saisie = Console.ReadLine();
n = Convert.ToInt32(saisie);
Console.WriteLine(n);
```



IV. Les Opérateurs :

+ - * / %	Addition Soustraction Multiplication Division modulo	<pre>int x = 7, y = 5; Console.WriteLine(x + y); //12 Console.WriteLine(x - y); //2 Console.WriteLine(x * y); //35 Console.WriteLine(x / y); //1 Console.WriteLine(x % y); //2 Console.ReadLine(); //Cette instruction attend une entrée</pre>
== != > <>= <=	Égalité Différence Supérieur Inférieur Sup ou égal Inf ou égal	<pre>int a = 3, b = 8; Console.WriteLine(a > b); //False Console.WriteLine(a < b); //True Console.WriteLine(a != b); //True Console.WriteLine(a == b); //False</pre>
=	Affectation ou assignation	<pre>int a; // on déclare un entier a. Console.WriteLine(a = 3); //assigne la valeur "3" à la variable "a".</pre>
+= -= *= /=	Opération puis réassignation	<pre>Console.WriteLine(a += 1); //a = a+1, donc a = 4. Console.WriteLine(a -= 2); //a = a-2, donc a = 2. Console.WriteLine(a *= 3); //a = a*3, donc a = 6. Console.WriteLine(a /= 6); //a = a/6, donc a = 1. Console.WriteLine(a %= 2); //a = a%2, donc a = 1.</pre>
 && ? ??	OU, ET Condition ? val_si_vrai : val_si_faux Variable référence ?? valeur si null	<pre>int a = 3, b = 6, c = 1; string d = null; Console.WriteLine(((a == 2) (a == 5))); //False Console.WriteLine(((b >= 0) && (b <= 10))); //True Console.WriteLine(((c == 1) ? "c est égal à 1" : "c n'est pas égal à 1")); //enregistre la première chaîne car c==1 Console.WriteLine(d ?? "d est null"); //enregistre d est null car d==null</pre>
var++, var-- , ++var, -- var	+1 ou -1 après utilisation de la variable. +1 ou -1 avant utilisation de la variable.	<pre>int a = 0; Console.WriteLine(a++); //affiche a, soit 0, puis l'incrémente. Console.WriteLine(a); //affiche la valeur courante de a, soit 1. Console.WriteLine(++a); //incrémente a, puis affiche sa valeur, soit 2.</pre>

V. Méthode Objet String :

Méthode	Description
Replace	Elle remplace toutes les occurrences d'un caractère dans la chaîne par un autre <code>cdc.Replace(char oldChar, char newChar);</code>
Split	Elle sépare la chaîne en plusieurs en fonction d'un caractère délimiteur. <pre>string phrase = "jumps over the lazy dog."; string[] words = phrase.Split(' ');</pre>
Substring	Retourne une partie de la chaîne. <code>Substring(Init32)</code> ou <code>Substring(Init32, Init32)</code>
ToCharArray	Retourne un tableau de type char à partir de la chaîne. <pre>string str = "012wxyz789";</pre>



```
char[] arr = str.ToCharArray(3, 4);
```

ToLower	Convertit tous les caractères de la chaîne en minuscules. <code>cdc.ToLower()</code>
ToUpper	Convertit tous les caractères de la chaîne en majuscules. <code>cdc.ToLower()</code>
Trim	Supprime les espaces en début et fin de chaîne. <code>cdc.Trim()</code>
classe StringBuilder	<pre>StringBuilder address = new StringBuilder(); address.Append("Bonjour"); address.Append(", Ahmed"); string concatenatedAddress = address.ToString(); Console.WriteLine(concatenatedAddress);</pre>

VI. L'Objet tableau (array) :

a. Tableau unidimensionnel :

```
int[] valeurs = new int[10];
int[] valeurs = new int[] {1, 2, 3, 4, 5};
string[] jours = new string[] { "Lundi", "Mardi", "Mercredi", "Jeudi", "Vendredi", "Samedi", "Dimanche" };
Lecture : Console.WriteLine(valeurs [0]);
Ecriture : valeurs [2]=50;
```

b. Tableau multidimensionnel :

```
int [, ] m = new int[10, 20];
string[, ] nom = new string[3, 2] { { "Mohamed", "Ali" }, { "Ahmed", "Ali " }, { "Aicha", "Ali " } };
```

c. jagged array ou bien tableau déchiqueté : ou "tableau de tableaux"

```
int[][] x = new int[10][];
x[0] = new int[11];
x[1] = new int[] { 2, 3 };
x[1][1] = 4;
```

Méthode sur tableau :

Length	<code>t.length</code> : retourne le nombre d'élément dans un tableau
Rank	<code>t.rank</code> : le nombre de dimensions dans le tableau
Clear()	<code>Array.Clear(array, index, length)</code> <code>Array.Clear(t, 1, 2);</code> rendre les valeur 0 de l'indice 1 à 2.
Copy()	<code>Array.Copy(sourceArray, destinationArray, sourceArray.Length)</code>
Find()	<code>Array.Find(t, match);</code> <code>Array.Find(numbers, number => number % 2 == 0);</code>
Sort()	<code>Array.sort(t)</code> : trier les valeurs d'un tableau t

Enumérations :

```
enum Automne { Octobre, Novembre, Décembre }
Automne moisAutomne;
moisAutomne = Automne.Octobre;
```

VII. Les Collections

a. Listes:

```
List<int> nombre = new List<int>(); // création de la liste
nombre.Add(10); // ajout du nombre 10 à la liste
nombre.Add(7); // ajout du nombre 7 à la liste
```



```

nombre.Add(5); // ajout du nombre 5 à la liste
nombre.RemoveAt(1);
List<string> jours = new List<string>();
jours.Add("Lundi"); // ajoute Lundi à la liste (indice 0).
jours.Add("Mardi");
jours.Add("Mercredi");
jours.Add("Jeudi");
jours.Add("Vendredi");
jours.Add("Samedi");
jours.Add("Dimanche");
int indice = jours.IndexOf("Samedi"); // indice vaut 5
jours.Remove("Mardi");
List<string> jours = new List<string> { "Lundi", "Mardi", "Mercredi", "Jeudi", "Vendredi",
"Samedi", "Dimanche" };

```

ArrayList

classe implémente un tableau d'objets de taille dynamique - Admet des méthodes permettant l'ajout, l'insertion et la suppression. - Admet Trois constructeurs :

```

using System.Collections;
// ...
ArrayList liste;
liste = new ArrayList();
liste = new ArrayList(capacité);
liste = new ArrayList(collection);
liste = new ArrayList(valeur1, ..., valeurN);

```

Affectation possible si la liste est initialisée

liste[indice] = élément;

Les méthodes de ArrayList :

entier = liste.Count;	Retourne le nombre des éléments de la liste
liste.Add(élément);	Ajoute un élément à la fin de la liste
liste.AddRange(collection);	Ajoute une collection à la fin de la liste
liste.Insert(index, élément);	Insère un élément dans la liste à l'index indiqué
liste.Remove(élément);	Supprime un élément de la liste
liste.RemoveAt(index);	Supprime l'élément à l'index indiqué de la liste
liste.Clear();	Supprime tous les éléments de la liste
object[] tableau; tableau = liste.ToArray();	Type "Object" obligatoire. Copie les éléments d'une liste dans un tableau
liste.Sort();	Tri d'une liste par ordre croissant ou alphabétique.
liste.Reverse();	Inverse l'ordre des éléments de la liste
booléen = liste.Contains(élément);	Vérifie l'existence d'un élément dans la liste.
entier = liste.IndexOf(élément); entier = liste.BinarySearch(valeur);	Retourne l'index de l'élément ou -1. Recherche dans une liste triée.
liste.Capacity = entier; entier = liste.Capacity;	Modifie le nombre d'éléments que la liste peut contenir. Retourne le nombre d'éléments que la liste peut contenir.

HashTable

Cette classe implémente une table de hachage de taille dynamique d'objets que l'on pourra retrouver plus tard sur la base d'une clé qui doit être unique.

Propose des méthodes permettant l'ajout ou la suppression d'un élément à la fois Elle possède plusieurs constructeurs

```

using System.Collections;
// ...
Hashtable ht;
ht = new Hashtable();
ht = new Hashtable(capacité);
ht = new Hashtable(hashtable);
ht = new Hashtable(storedList);

```

Affectation possible si la table de hachage n'est pas vide.

ht[clé] = élément;

Cast obligatoire si le type n'est pas Object.

Type élément = (Type) ht[clé];

Les méthodes utilisées :

entier = ht.Count;	Retourne le nombre des éléments de la table de hachage.
ht.Add(clé, élément); ht.Remove(clé); ht.Clear();	- Ajoute un élément avec sa clé à la fin de la table de hachage. - Supprime un élément de la table de hachage - Supprime tous les éléments de la table de hachage
booléen = ht.Contains(clé); booléen = ht.ContainsKey(clé); booléen = ht.ContainsValue(élément);	- Vérifie l'existence d'une clé dans la table de hachage. - Vérifie l'existence d'un élément dans la table de hachage.
Type[] tableau; ht.Values.CopyTo(tableau, indice_tableau);	- Type des éléments de la table de hachage - Copie les éléments de la table de hachage vers un tableau spécifique d'objets à l'indice spécifié.
Type[] tableau; ht.Keys.CopyTo(tableau, indice_tableau);	Type des clés de la table de hachage. Copie les clés de la table de hachage vers un tableau spécifique d'objets à l'indice spécifié.
object[] tableau; ht.CopyTo(tableau, indice_tableau);	- Type "object" obligatoire - Copie les clés et les éléments de la table de hachage vers un tableau d'objets à l'indice spécifié

d. SortedList<TKey,T>

Les classes "SortedList" et "SortedList<TKey,T>" implémentent une liste triée selon une clé et sans possibilité de doublons, ce qui implique que toutes les clés doivent être différentes

Elle possède plusieurs constructeurs

```

using System.Collections;
// ...
SortedList sl;
sl = new SortedList();
sl = new SortedList(capacité);
sl = new SortedList(hashtable);
sl = new SortedList(storedList);

```

Les méthodes utilisées

sl[clé] = élément;	Affectation possible si la liste triée n'est pas vide.
Type élément = (Type) sl[clé];	Cast obligatoire si le type n'est pas Object.

<code>Type élément = (Type)</code> <code>sl.GetByIndex(indice);</code>	Retourne l'élément par son indice.
<code>Type clé = (Type) sl.GetKeyList()[indice];</code>	Retourne la clé par son indice.
<code>Type élément = (Type)</code> <code>sl.GetValueList()[indice];</code>	Retourne l'élément par son indice.
<code>entier = sl.Count;</code>	Retourne le nombre des éléments de la liste triée.
<code>entier = sl.IndexOfKey(clé);</code> <code>entier =</code> <code>sl.IndexOfValue(élément);</code>	- Retourne l'indice d'une clé - Retourne l'indice d'un élément
<code>sl.Add(clé, élément);</code> <code>sl.Remove(clé);</code> <code>sl.RemoveAt(indice);</code> <code>sl.Clear();</code>	- Ajoute un élément avec sa clé à la fin de la liste triée - Supprime un élément de la liste triée - Supprime un élément de la liste triée à l'indice spécifié - Supprime tous les éléments de la liste triée
<code>booléen = sl.Contains(clé);</code> <code>booléen =</code> <code>sl.ContainsKey(clé);</code> <code>booléen =</code> <code>sl.ContainsValue(élément);</code>	- Vérifie l'existence d'une clé dans la liste triée - Vérifie l'existence d'un élément dans la liste triée
<code>Type[] tableau;</code> <code>sl.Values.CopyTo(tableau, indice_tableau);</code>	- Type des éléments de la liste triée. - Copie les éléments de la liste triée vers un tableau spécifique d'objets à l'indice spécifié.
<code>Type[] tableau;</code> <code>sl.Keys.CopyTo(tableau, indice_tableau);</code>	- Type des clés de la liste triée. - Copie les clés de la liste triée vers un tableau spécifique d'objets à l'indice spécifié.
<code>object[] tableau;</code> <code>sl.CopyTo(tableau, indice_tableau);</code>	- Type "object" obligatoire. - Copie les clés et les éléments de la liste triée vers un tableau d'objets à l'indice spécifié.

VIII. La classe Datetime :

a. La syntaxe :

`DateTime( int année, int mois, int jour);`

`DateTime( int a, int m, int j, int heure, int minutes, int secondes) ;`

`DateTime( int a, int m, int j, int heure, int minutes, int secondes, int millisec); -`

`DateTime(long);`

b. Les propriétés de la classe datetime :

`Date – Day – DayOfWeek – DayOfYear- Hour – Millisecond - Minute - Month - Now - Second - Ticks -`

`TimeOfDay - Today - UtcNow – Year.`

c. Les méthodes de la classe datetime :

`DateTime Add(TimeSpan interv);`

`DateTime AddDays(double n);`

`int Compare( DateTime t1, DateTime t2) ;`

`int CompareTo(DateTime t);`

`int DaysInMonth( int année, int mois);`

`string ToString(string format) ;`

`DateTime Parse(string s) ;`

`bool IsLeapYear(int années) ;`

TP à réaliser